

Bicol University
College of Science – Information Technology Department
IT 112 – Web Systems and Technologies

Laboratory Report: Phase 6 – Edit and Delete Functionality

Group Name: Code Breakers – BSIT 2B
GitHub Repository: final-project-Code-breakers-BSIT2b
Branch: main / phase-6
Date Submitted: May 13, 2026

Step 1: Backend – PUT and DELETE Routes (userRoutes.js)

The backend routes file was updated to include two new endpoints: a PUT route for updating a user by ID, and a DELETE route for removing a user by ID. Both routes use Mongoose methods and include error handling.

Step 2: Frontend – Edit and Delete Buttons (admin-dashboard.html)

Each user row in the admin dashboard table was updated to include an Edit button (triggering a modal) and a Delete button (calling the deleteUser function directly).

Step 3: Edit Function (admin.js)

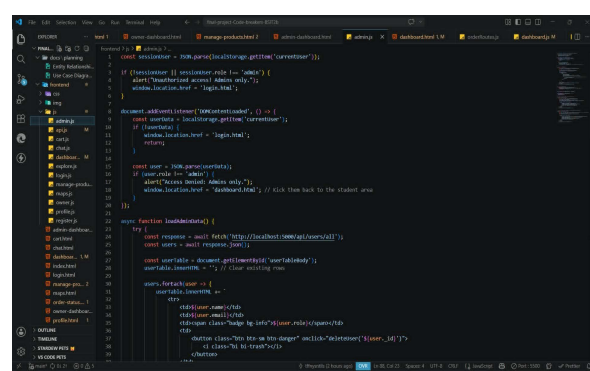
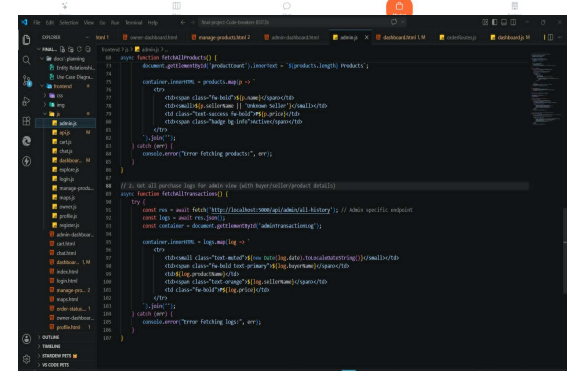
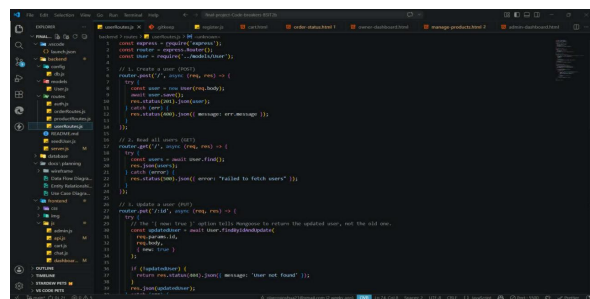
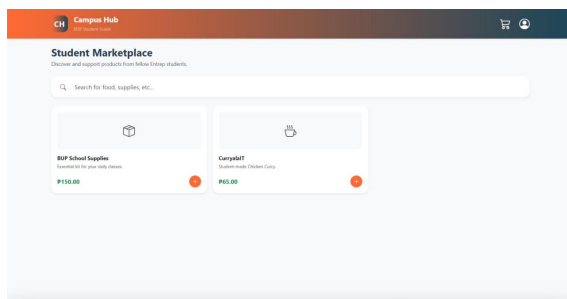
The editUser function uses a browser prompt to collect the new name, then sends a PUT request to the backend API. On success, the user list is refreshed.

Step 4: Delete Function (admin.js)

The deleteUser function shows a confirmation dialog before sending a DELETE request. After deletion, the admin data is reloaded to reflect the change.

Step 5: GitHub Repository Update

- All modified files (userRoutes.js, admin.js, admin-dashboard.html) were committed.
- Commit message: 'Complete Phase 6: Edit and Delete functionality'
- Changes were pushed to the main branch of the repository.
- Repository: github.com/wahtoshiii/final-project-Code-breakers-BSIT2b



```

1 // ...
2 router.get('/:id', async (req, res) => {
3   const {id} = req.params;
4   res.status(200).json({ message: 'user not found' });
5 });
6
7 // ...
8 // 4. Delete a user (DELETE)
9 router.delete('/:id', async (req, res) => {
10   try {
11     const {id} = req.params;
12     await user.findByIdAndDelete(id);
13     res.status(200).json({ message: 'user removed successfully' });
14   } catch (error) {
15     res.status(404).json({ error: 'failed to delete user' });
16   }
17 });
18
19 // ...
20 // 5. Add a new user (POST)
21 router.post('/', async (req, res) => {
22   try {
23     const {username, password} = req.body;
24     if (!username || !password) {
25       res.status(400).json({ error: 'username and password are required' });
26     }
27     const newUser = new User({ username, password });
28     await newUser.save();
29     res.status(201).json(newUser);
30   } catch (error) {
31     res.status(400).json({ error: 'invalid user data' });
32   }
33 });
34
35 // ...
36 module.exports = router;

```

```

1 // ...
2 // 6. Update a user (PUT)
3 router.put('/:id', async (req, res) => {
4   try {
5     const {id} = req.params;
6     const {username, password} = req.body;
7     const user = await User.findById(id);
8     if (!user) {
9       res.status(404).json({ error: 'user not found' });
10     }
11     user.username = username;
12     user.password = password;
13     await user.save();
14     res.status(200).json(user);
15   } catch (error) {
16     res.status(400).json({ error: 'invalid user data' });
17   }
18 });
19
20 // ...
21 // 7. Get all users (GET)
22 router.get('/', async (req, res) => {
23   try {
24     const users = await User.find();
25     res.status(200).json(users);
26   } catch (error) {
27     res.status(500).json({ error: 'failed to fetch users' });
28   }
29 });
30
31 // ...
32 // 8. Get a single user (GET)
33 router.get('/:id', async (req, res) => {
34   try {
35     const {id} = req.params;
36     const user = await User.findById(id);
37     if (!user) {
38       res.status(404).json({ error: 'user not found' });
39     }
40     res.status(200).json(user);
41   } catch (error) {
42     res.status(500).json({ error: 'failed to fetch user' });
43   }
44 });
45
46 // ...
47 // 9. Get all products (GET)
48 router.get('/products', async (req, res) => {
49   try {
50     const products = await Product.find();
51     res.status(200).json(products);
52   } catch (error) {
53     res.status(500).json({ error: 'failed to fetch products' });
54   }
55 });
56
57 // ...
58 // 10. Get a single product (GET)
59 router.get('/products/:id', async (req, res) => {
60   try {
61     const {id} = req.params;
62     const product = await Product.findById(id);
63     if (!product) {
64       res.status(404).json({ error: 'product not found' });
65     }
66     res.status(200).json(product);
67   } catch (error) {
68     res.status(500).json({ error: 'failed to fetch product' });
69   }
70 });
71
72 // ...
73 // 11. Create a new product (POST)
74 router.post('/products', async (req, res) => {
75   try {
76     const {name, price} = req.body;
77     if (!name || !price) {
78       res.status(400).json({ error: 'name and price are required' });
79     }
80     const newProduct = new Product({ name, price });
81     await newProduct.save();
82     res.status(201).json(newProduct);
83   } catch (error) {
84     res.status(500).json({ error: 'failed to create product' });
85   }
86 });
87
88 // ...
89 // 12. Update a product (PUT)
90 router.put('/products/:id', async (req, res) => {
91   try {
92     const {id} = req.params;
93     const {name, price} = req.body;
94     const product = await Product.findById(id);
95     if (!product) {
96       res.status(404).json({ error: 'product not found' });
97     }
98     product.name = name;
99     product.price = price;
100     await product.save();
101     res.status(200).json(product);
102   } catch (error) {
103     res.status(500).json({ error: 'failed to update product' });
104   }
105 });
106
107 // ...
108 // 13. Delete a product (DELETE)
109 router.delete('/products/:id', async (req, res) => {
110   try {
111     const {id} = req.params;
112     const product = await Product.findById(id);
113     if (!product) {
114       res.status(404).json({ error: 'product not found' });
115     }
116     await product.delete();
117     res.status(200).json({ message: 'product deleted successfully' });
118   } catch (error) {
119     res.status(500).json({ error: 'failed to delete product' });
120   }
121 });
122
123 // ...
124 module.exports = router;

```

